

Week 6 Lab: Employee Management System (Functions & Basic Calculations)

Introduction

In phase 1 of this second project you create a **Employee Management System** program that processes employee payroll records, calculates compensation with overtime, and generates summary reports.

What you'll build in Phase 1:

- A program that processes multiple employee time records
- Functions that calculate gross pay (with overtime), deductions, and net pay
- A loop that allows multiple employee entry
- Running totals that summarize all employees processed

LEARNING OBJECTIVES

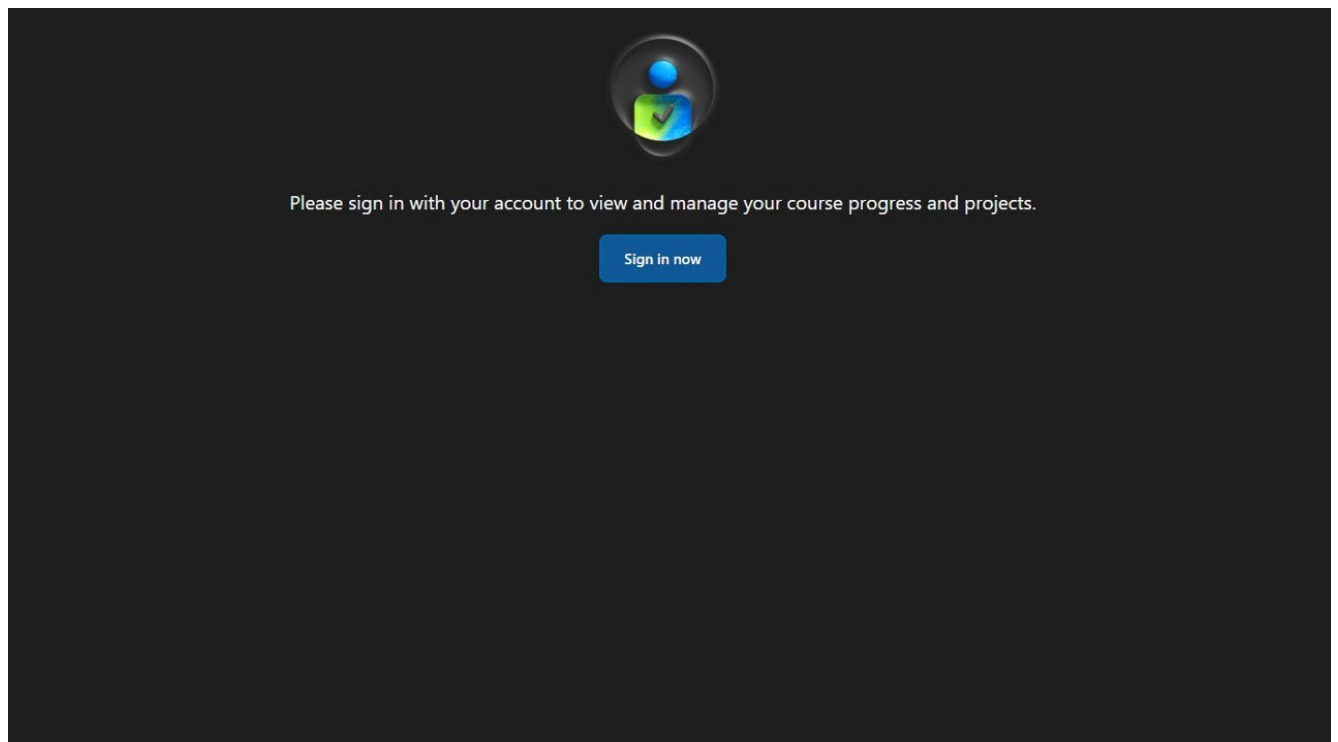
By completing this phase, you will demonstrate your ability to:

- Create functions that accept parameters and return values
- Use while loops to process multiple records
- Implement loop termination with user input
- Accumulate running totals across iterations
- Display formatted output with proper labels
- Apply the 25% deduction rule to calculate net pay
- Calculate overtime pay ($1.5\times$ rate for hours over 40)

Lab Instructions

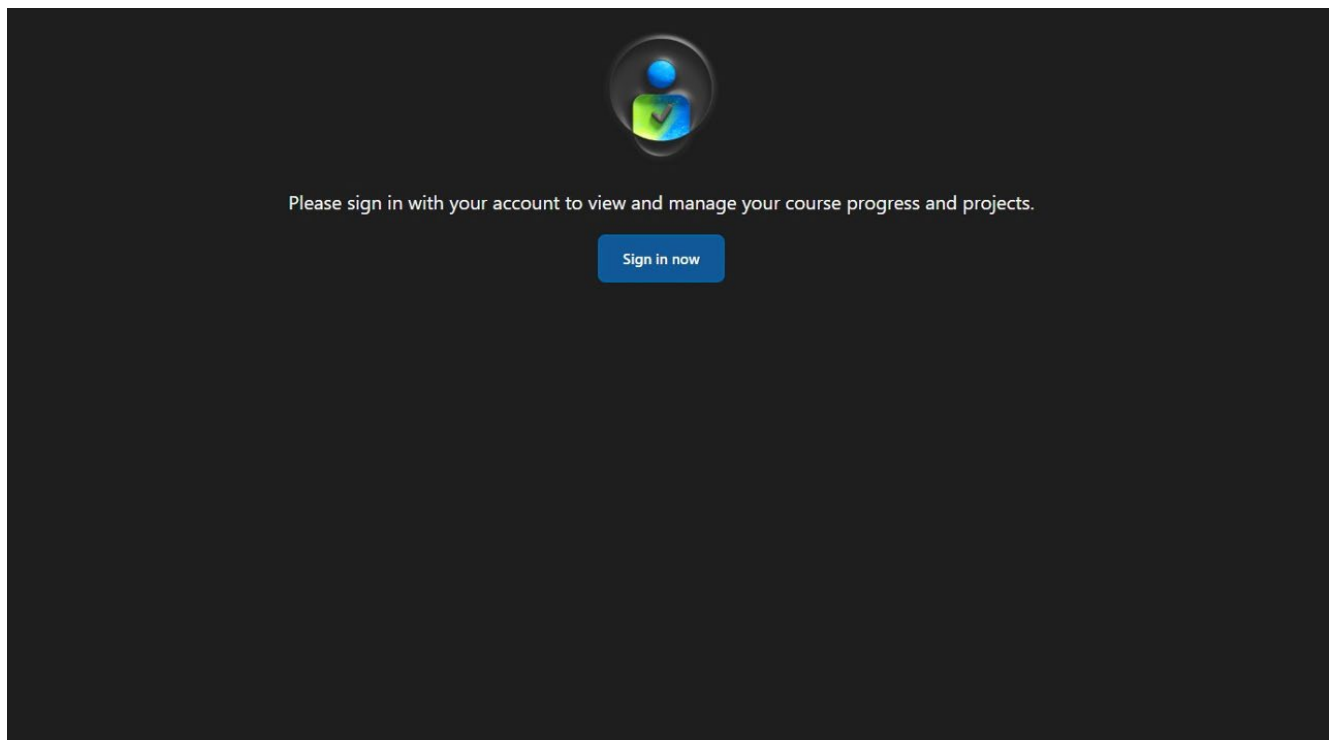
Step 1

Click on <https://vscodeedu.com/my-work/projects> to access Visual Studio Code for Education.

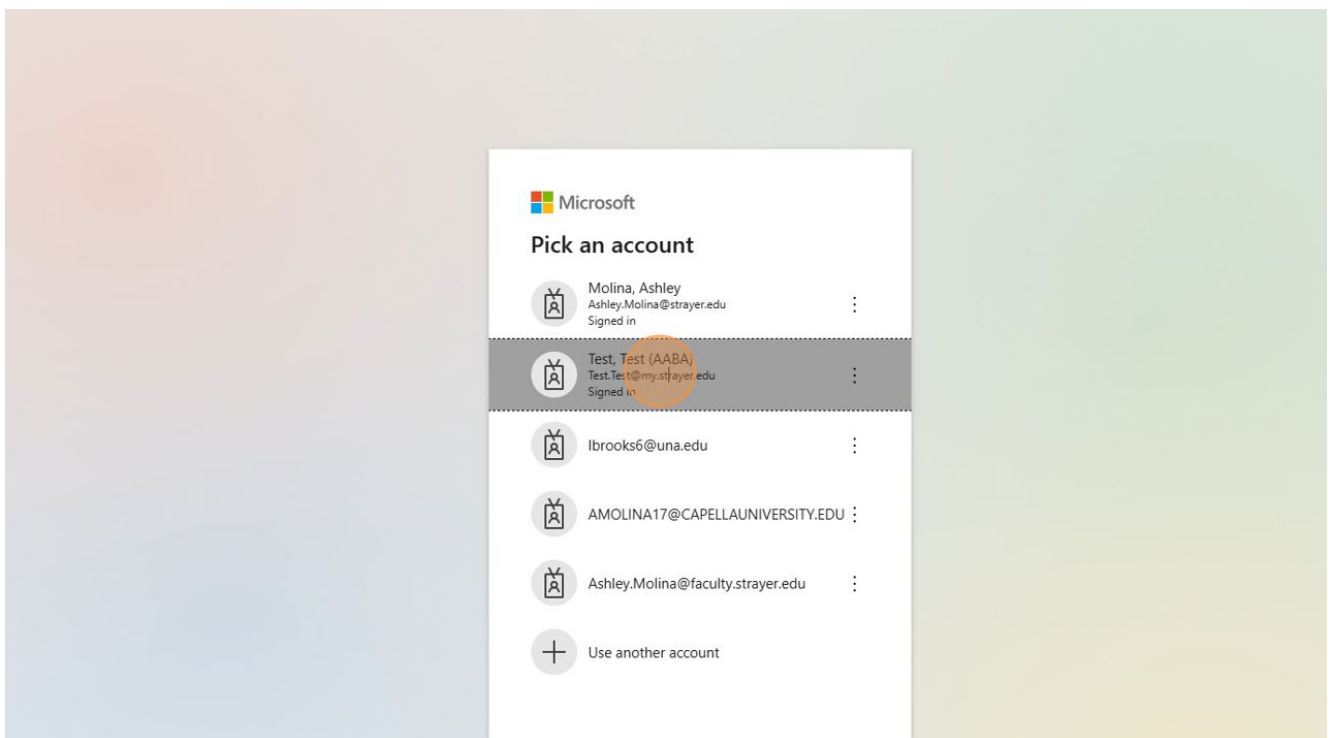
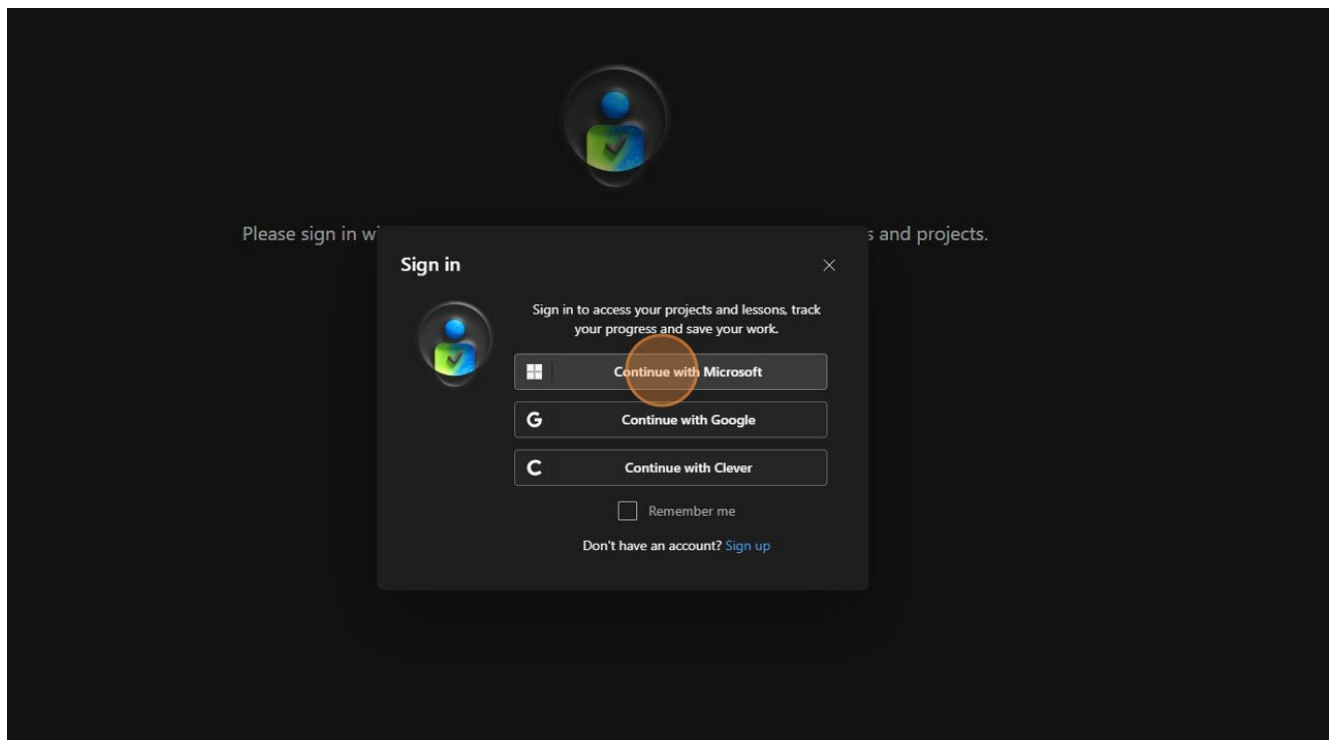


Step 2

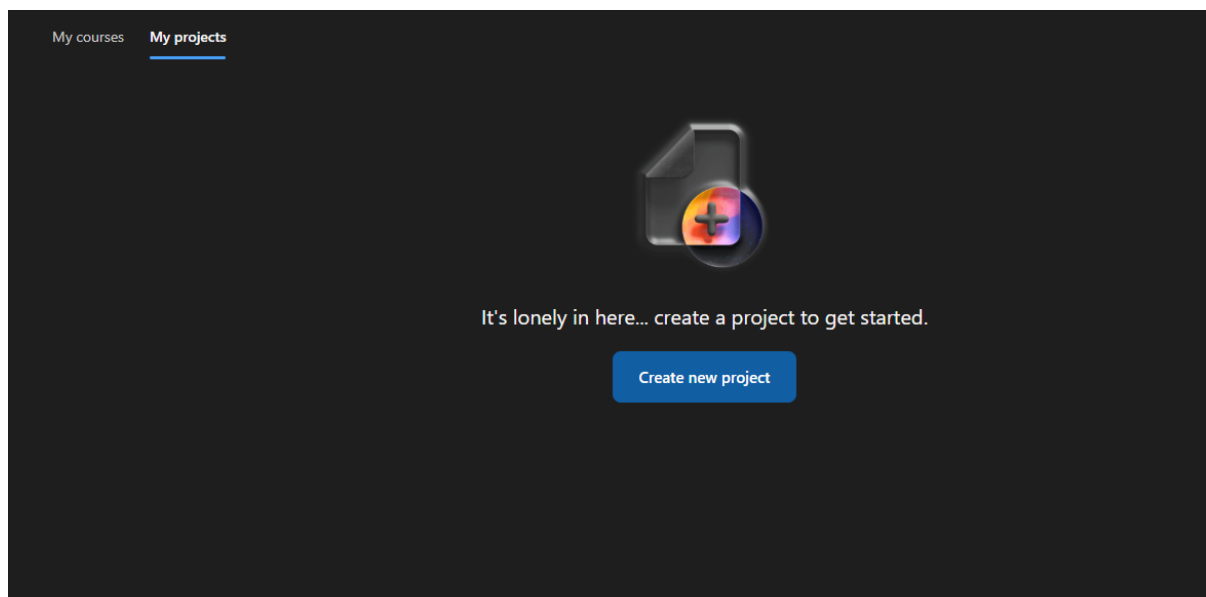
You will then Click 'Sign in now'.



Then select **'Continue with Microsoft' or 'Continue with Google'** to sign in with your already established email address. Using 'Continue with Microsoft' you will sign in with your Strayer email address that ends in @my.strayer.edu and your iCampus password. You will be prompted to log into the Strayer Systems as normal.



Step 3



Step 4

Create a new project named **Employee Management System**.

Step 5

Add your name, course number, lab title, and Phase as comments at the start of the project.

```
1  #Your Name
2  #CIS261
3  #Employee Management System
4  #Phase 1
```

Step 6

Write a program to perform the following functionality.

Part 1

Create the following input functions. Each function should prompt the user for information and return the appropriate value(s):

- **get_employee_name()** - Returns first name and last name. If user types "ESC" for first name, return "ESC" for first name.
- **get_employee_id()** - Returns employee ID as a string.
- **get_hours_worked()** - Returns hours worked as a float.
- **get_hourly_rate()** - Returns hourly rate as a float.

Part 2

Create the following calculation functions:

- **calculate_gross_pay(hours, rate)** - Calculates gross pay. If hours > 40, pay overtime at $1.5 \times$ rate for hours over 40. Returns gross pay.
- **calculate_deductions(gross_pay)** - Calculates total deductions as 25% of gross pay. Returns deductions.
- **calculate_net_pay(gross_pay, deductions)** - Calculates net pay as gross pay minus deductions. Returns net pay.

Part 3

Create the following display functions:

- **display_employee_record(first_name, last_name, employee_id, hours, rate, gross_pay, deductions, net_pay)** - Displays formatted payroll information for one employee including all input values and calculated results.
- **display_summary(total_employees, total_gross, total_deductions, total_net)** - Displays summary totals for all employees processed.

Part 4

Create a **main()** function that implements the following functionality:

- Display a welcome message with instructions. Include information about overtime calculation ($1.5 \times$ rate for hours > 40) and deductions (25% total).
- Initialize four counters before the loop: **total_employees, total_gross, total_deductions, total_net**.
- Use a while loop to:
 - Call the input functions to get employee information.
 - Check if user typed "ESC" (case-insensitive) and break the loop if so.
 - Call the calculation functions to compute gross pay, deductions, and net pay.

- Call the display function to show the employee record.
- Accumulate the totals (add 1 to employee count, add amounts to running totals).
- After the loop ends, if employees were processed, call the summary display function.
- Call **main()** at the end of the program to run it.

Part 5

Use the following rules for calculations:

- **Gross Pay:** If $\text{hours} \leq 40$, $\text{gross} = \text{hours} \times \text{rate}$. If $\text{hours} > 40$, $\text{gross} = (40 \times \text{rate}) + ((\text{hours} - 40) \times \text{rate} \times 1.5)$.
- **Deductions:** $\text{Total deductions} = \text{gross_pay} \times 0.25$ (represents 25% for Federal 15%, State 5%, Benefits 5%).
- **Net Pay:** $\text{net_pay} = \text{gross_pay} - \text{deductions}$.

NOTE: Be sure to display the employee record each time an employee is processed, and display the summary after all employees are entered.

Step 7

Write and test the code for the lab.

Step 8

Testing Requirements

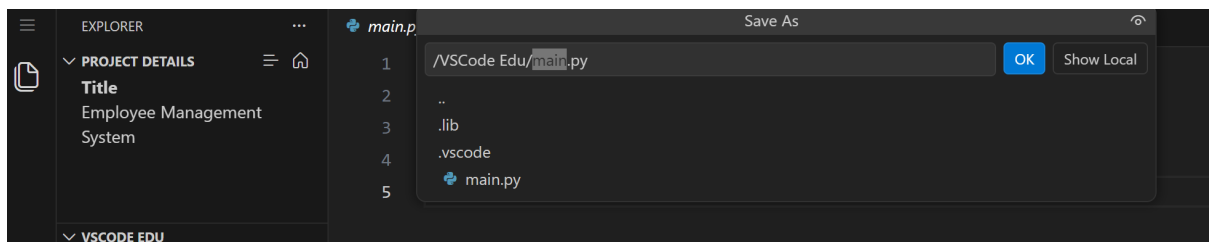
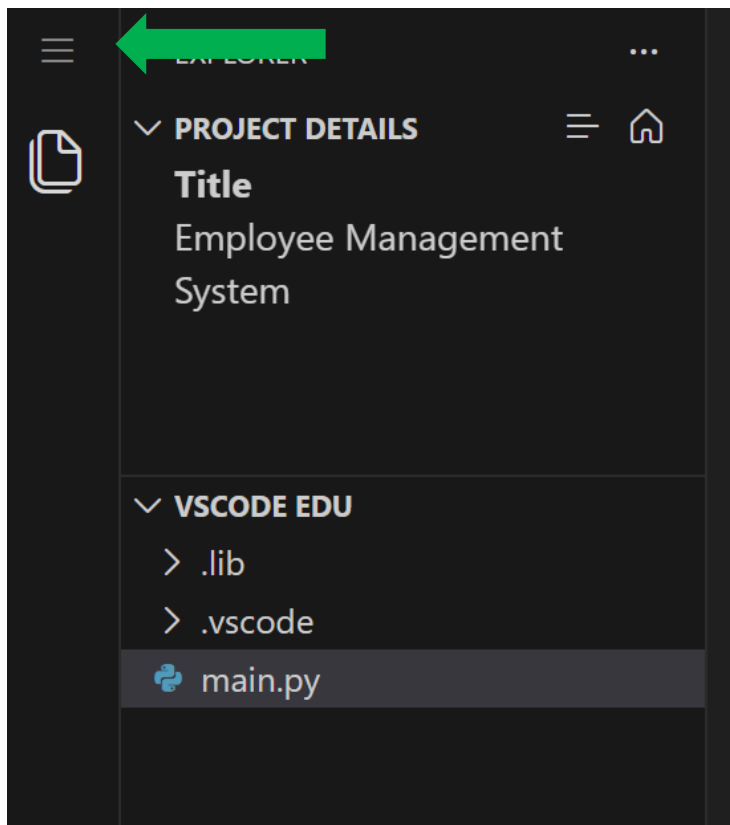
Test your program with a minimum of **2 employees** to demonstrate:

- An employee with 40 or fewer hours (no overtime)
- An employee with more than 40 hours (overtime calculated)
- Proper ESC termination (test both "ESC" and "esc")
- Correct calculation of deductions (exactly 25% of gross pay)
- Accurate summary totals

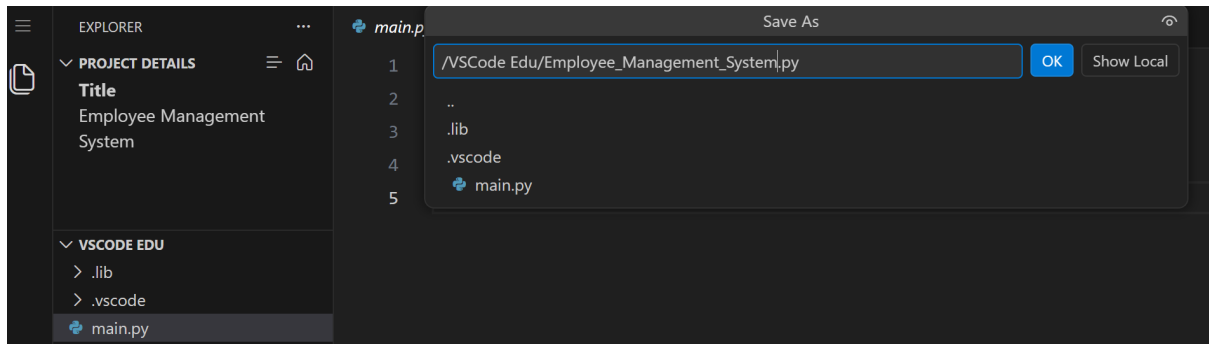
Saving Your Work to Your Computer

Now, we are going to rename and download our python file for submission!

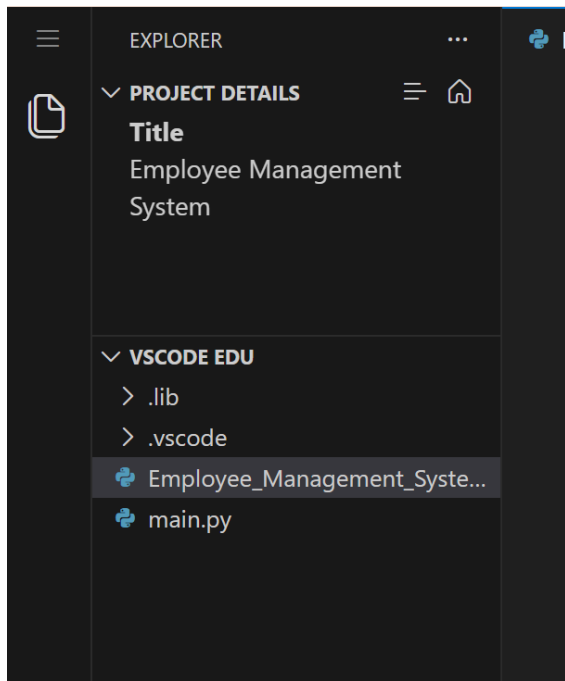
To rename your file, click on the 3 dashes in the top left corner of Visual Studio Code for Education. Then select File → Save As → This will open up a window in the top middle of your screen with main highlighted.



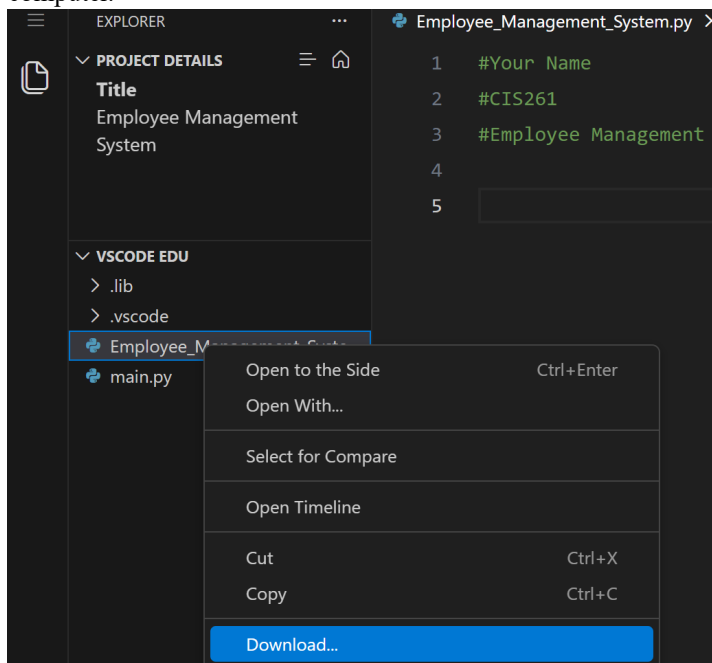
Now, let's replace main with **Employee_Management_System** – **Do Not Delete the .py**. Click **Ok** to save the file with the new name to your project.



You will now see both the **main.py** and the **Employee_Management_System.py** file in your VSCODE EDU area.



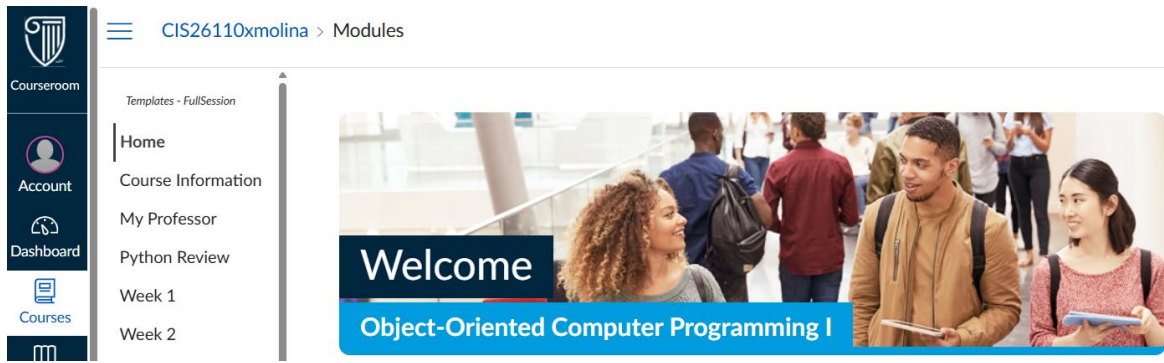
We need to download your **Employee_Management_System.py** file to your computer to prepare your submission. To do this right-click on **Employee_Management_System.py** and a pop-up box will appear. Select **Download** from the menu, to download the file to your downloads folder on your computer.



If this is the first time in here you may get a pop-up that asks 'vscode.dev wants to See text and images copied to the clipboard Block or Allow' click Allow. Once you 'Allow' you will then see your menu to download your file.

Now your python file for this project is in your computer's downloads folder!

Go to <https://canvas.strayer.edu/> in your web browser, sign-in, and select **CIS261 Object-Oriented Programming** from your tiles.



Step 3

Select Week 6 – Lab from the Week 6 menu options

For this lab you should submit your **CIS261_Employee_Management_System.py** directly to the **Week 6 – Lab** submission area.

Step 4

Select the **Upload** option and then select **Choose a file to upload** from the submission area and select **CIS261_Employee_Management_System.py** from your selected location in your File Explorer.

When you select the file you will click **Open** to add the file to Canvas.

Once your file shows a green check mark beside it click ***“I agree to the tool’s End-User License Agreement. This assignment submission is my own, original work”*** and click **“Submit Assignment”**

Review

Well, done, you have completed the **First Step of the Employee Management System!**

Important Notes

- Loop must terminate when user types "ESC" (case-insensitive)
- All dollar amounts must display with exactly 2 decimal places (use `:.2f`)
- Hours should display with 1 decimal place (use `:.1f`)
- Overtime is calculated **ONLY** on hours over 40 (not all hours)
- Deductions are exactly 25% of gross pay
- All four totals must accumulate correctly throughout the loop
- Summary should only display if at least one employee was processed

END OF DOCUMENT